Disentangling Code Exposure from Model Diversity in Automated LLM Test Generation: A 2×2 Factorial Empirical Study

Ziad Emad Allam

Independent Researcher, Benha, Egypt
ziademadbts@gmail.com

November 4, 2026

Abstract

Context: Recent empirical investigations into Large Language Model (LLM) test generation report conflicting findings: exposing test generators to code impairs defect detection ("code-first degradation"), yet pairing different models under white-box code visibility yields negligible improvements. **Objective:** We conduct a 2×2 factorial crossover experiment to disentangle the effect of Code Exposure (Specification-Only vs. Code-First) from Model Diversity (Same Coder vs. Cross-Family Architecture). **Method:** Evaluating $N = 225$ matched trial quadruplets (900 recorded condition evaluations across 15 verified, organic Python logic defects harvested from Llama 3.3 70B, Qwen 2.5 Coder 32B, and Qwen 2.5 7B), we analyze defect detection using a hierarchical Generalized Linear Mixed Model (GLMM) with problem random intercepts, supported by paired exact tests. **Results:** The primary protocol GLMM confirms a statistically significant negative interaction between code exposure and model diversity ($\beta = -1.0358, p = 0.0077, \text{OR} = 0.355$, 95% CI: $[0.166, 0.760]$; Likelihood-Ratio Test $p = 0.0072$). In trial-level comparisons, code exposure reduces defect detection from 33.78% to 24.44% for the same model ($\Delta = -9.33$ percentage points, exact McNemar $p = 0.0046$). Under code exposure, cross-family diversity yields no significant gain (24.44% vs. 18.22%, $p = 0.0649$), and equivalence within an 8% margin is not demonstrated ($p_{\text{TOST}} = 0.2841$). Descriptively, this drop is accompanied by reduced test suite usability (86.2% to 64.9%), with conditional detection rates among usable suites being closer (39.18% vs. 37.67%). **Conclusion:** Within the evaluated tasks and model configurations, the benefit of model diversity depends critically on prompt exposure: cross-model diversity achieves peak detection under specification-only prompts (40.00%), but this advantage vanishes under code exposure.

Keywords: LLM Test Generation, Code-First Bias, Model Diversity, Empirical Software Engineering, Generalized Linear Mixed Models (GLMM), Oracle Problem, Information and Software Technology.

1 Introduction

Automated unit test generation using Large Language Models (LLMs) has emerged as an active area of empirical software engineering research [2, 6, 10]. In contrast to traditional search-based software testing (SBST), generative LLMs synthesize human-readable assertions and execution inputs directly from natural language specifications and existing source code.

However, two recent empirical investigations have raised critical, unresolved questions regarding the configuration of LLM test generation workflows:

1. **Code-First Exposure Degradation:** Konstantinou, Tambon, and Papadakis [4] evaluated multiple foundation models across HumanEval+ and BigCodeBench, finding that

prompting an LLM with an existing implementation severely degrades the fault-detection capability of the generated test suites relative to prompting with specifications alone (a drop from 18–33% down to 5–17%). This phenomenon has been described as a code-first degradation in test oracle efficacy.

2. **Invariance to Model Diversity under Code Exposure:** Conversely, Leu, Oertel, and Hebig [7] evaluated whether deploying an independent, distinct LLM family to test code authored by another model could mitigate oracle deficiencies. In an empirical study on Elixir, they found no statistically significant difference between same-model and different-model test generators ($p = 0.152$), concluding that architectural diversity did not yield detectable benefits.

Crucially, these prior studies varied distinct operational dimensions. While Konstantinou et al. [4] examined code exposure within single-model workflows, Leu et al. [7] evaluated model diversity strictly in a white-box setting (where the implementation was always visible to the test generator), lacking a specification-only baseline. Consequently, it remains an open question whether model diversity is fundamentally ineffective, or whether the presence of implementation code overrides the potential gains of architectural diversity.

To resolve this empirical tension, we design a balanced 2×2 factorial crossover experiment. We isolate two independent experimental factors:

- **Factor A (Code Exposure):** Specification-Only (T_0, T_3) vs. Code-First (T_1, T_2).
- **Factor B (Model Identity):** Intra-Family Same Coder (T_0, T_1) vs. Cross-Family Evaluator (T_2, T_3).

Evaluating $N = 225$ matched trial quadruplets (900 condition evaluations) across 15 verified, organic Python logic defects harvested from modern open-weight LLMs (Llama 3.3 70B, Qwen 2.5 Coder 32B, and Qwen 2.5 7B), we address three specific research questions:

- **RQ1 (Code-First Effect):** Does exposing the test-generating model to implementation code decrease defect detection relative to generating tests from specifications alone (T_0 vs. T_1)?
- **RQ2 (Model Diversity & Equivalence under Code):** Holding code exposure constant, does employing an independent model family provide equivalent or superior defect detection compared to the authoring model (T_1 vs. T_2)?
- **RQ3 (Factorial Interaction):** Does the effect of cross-model diversity significantly depend on whether the model is exposed to implementation code, as estimated by a hierarchical Generalized Linear Mixed Model?

2 Benchmark Design & Defect Harvesting

2.1 Natural Logic Defects vs. Synthetic Mutants

A central consideration in empirical testing literature is the relationship between synthetic mutants and real-world software defects. While landmark empirical work by Just et al. [3] demonstrated that mutant detection is positively correlated with real fault detection, their findings also established that a non-trivial fraction of real faults (up to 17%) are not coupled to standard mutation operators, and that artificial mutants frequently model localized, syntactic perturbations rather than complex multi-statement algorithmic logic errors. Furthermore, empirical evaluations by Liu et al. [8] emphasize that code synthesized by LLMs frequently passes superficial syntax and sanity checks while harboring subtle edge-case logic defects that require semantically rich test oracles to expose.

Table 1: The 15 Verified Natural LLM Logic Defects (Generated Directly from `accepted_faults.json`).

Task ID	Full Function Name	Category	Difficulty	Kill Ratio (Fraction)	Authoring Model (Coder)	Temp.
REAL_TASK_001	parse_github_url	Parsing/String	Hard	16.7% (1/6)	meta-llama/llama-3.3-70b-instruct	$T = 0.8$
REAL_TASK_002	split_by_positions	Parsing/String	Medium	50.0% (2/4)	meta-llama/llama-3.3-70b-instruct	$T = 0.8$
REAL_TASK_004	validate_tags	Parsing/String	Hard	20.0% (1/5)	meta-llama/llama-3.1-8b-instruct	$T = 1.0$
REAL_TASK_005	split_host_port	Parsing/String	Medium	42.9% (3/7)	qwen/qwen-2.5-coder-32b-instruct	$T = 0.8$
REAL_TASK_006	get_consistent_hash_node	Algorithms/Search	Medium	50.0% (2/4)	meta-llama/llama-3.3-70b-instruct	$T = 1.0$
REAL_TASK_010	find_conflicting_reservations	Graphs/Trees	Medium	50.0% (2/4)	meta-llama/llama-3.3-70b-instruct	$T = 0.8$
REAL_TASK_011	check_half_open_overlap	Scheduling/Intervals	Hard	16.7% (1/6)	meta-llama/llama-3.3-70b-instruct	$T = 0.8$
REAL_TASK_013	intersect_intervals	Scheduling/Intervals	Hard	20.0% (1/5)	meta-llama/llama-3.3-70b-instruct	$T = 0.8$
REAL_TASK_014	compute_exponential_backoff_sequence	State Machines/Retries	Hard	33.3% (1/3)	meta-llama/llama-3.1-8b-instruct	$T = 0.9$
REAL_TASK_015	compute_fibonacci_backoff_sequence	Algorithms/Search	Hard	33.3% (1/3)	qwen/qwen-2.5-coder-32b-instruct	$T = 0.8$
REAL_TASK_016	evaluate_token_bucket	State Machines/Retries	Hard	14.3% (1/7)	meta-llama/llama-3.3-70b-instruct	$T = 0.8$
REAL_TASK_017	check_sliding_window_rate_limit	State Machines/Retries	Hard	11.1% (1/9)	meta-llama/llama-3.3-70b-instruct	$T = 0.8$
REAL_TASK_018	evaluate_circuit_breaker_transition	State Machines/Retries	Hard	22.2% (2/9)	meta-llama/llama-3.3-70b-instruct	$T = 0.8$
REAL_TASK_019	merge_overlapping_intervals	Scheduling/Intervals	Hard	25.0% (1/4)	meta-llama/llama-3.3-70b-instruct	$T = 0.8$
REAL_TASK_020	detect_dag_cycle_and_ancestors	Graphs/Trees	Medium	40.0% (2/5)	qwen/qwen-2.5-7b-instruct	$T = 0.7$

To examine test generation under ecologically valid conditions, our benchmark consists entirely of **15 natural, organic logic faults** produced by open-weight LLMs during functional code synthesis. The tasks were sampled from public GitHub repositories containing clean reference implementations (C_{ref}) and comprehensive golden unit test suites, spanning five distinct software categories: String Parsing, Algorithmic Search, Interval Scheduling, State Machines / Rate Limiting, and Graph Structures. A complete provenance manifest detailing each task's source repository, commit hash, author, and licensing status is provided in the replication package.

2.2 Defect Harvesting and Verification Protocol

Candidate faulty implementations (C_{faulty}) were harvested by prompting frontier open-weight models across various sampling temperatures ($T \in \{0.7, 0.8, 0.9, 1.0\}$). Each candidate was filtered using objective criteria:

1. **Syntactic Viability:** The implementation must be syntactically valid, compile cleanly, execute on standard inputs without unhandled runtime crashes, and preserve the exact signature and docstring.

2. **Empirical Defect Boundary:** The candidate must fail at least one assertion in the golden test suite, but no more than 50.0% of golden test cases (Kill Ratio $\leq 50.0\%$).

Table 1 lists the complete set of 15 verified benchmark tasks, generated directly from the archived dataset manifest (`accepted_faults.json`). Ten tasks represent *Hard* defects (kill ratio $\leq 33.4\%$) and five represent *Medium* defects ($33.4\% - 50.0\%$). In all 15 tasks, the authoring model was a member of the Llama or Qwen families; DeepSeek-V3 was employed exclusively as an independent cross-model evaluator during testing, rather than as a defect generator.

3 Experimental Methodology

3.1 2×2 Factorial Crossover Design

To evaluate code exposure and model diversity without confounding task difficulty, each trial evaluates an identical problem across four permuted conditions:

- T_0 **(Spec-Only, Same Model):** The authoring model generates tests provided solely with the docstring and function signature.

- T_1 **(Code-First, Same Model):** The authoring model generates tests provided with the docstring, signature, and its own faulty implementation (C_{faulty}).

- T_2 **(Code-First, Cross-Family Model):** An independent model family generates tests provided with the docstring, signature, and C_{faulty}.

- T_3 **(Spec-Only, Cross-Family Model):** The independent model family generates tests provided solely with the docstring and function signature.

3.2 Model Configuration, Prompting, and Reproducibility

To ensure reproducibility, all generation requests were executed via the OpenRouter unified API with a pre-registered global seed $S = 42$, sampling temperature $T = 0.2$, and a generation limit of max_tokens = 800. Evaluated open-weight models reflect exact pinned provider backends and precision configurations:

- **Coder Models (Defect Synthesis):**
  - `meta-llama/llama-3.3-70b-instruct` (DeepInfra, bf16)
  - `qwen/qwen-2.5-coder-32b-instruct` (DeepInfra, bf16)
  - `meta-llama/llama-3.1-8b-instruct` (DeepInfra, bf16)
  - `qwen/qwen-2.5-7b-instruct` (DeepInfra, bf16)
- **Cross-Family Evaluator Models:**
  - `deepseek/deepseek-chat` (DeepSeek-V3, fp8, hosted by DeepSeek) for Llama tasks.
  - `meta-llama/llama-3.3-70b-instruct` (DeepInfra, bf16) for Qwen-authored tasks.

Test generators received standardized prompt templates across all conditions:

- **Specification-Only Prompt (T_0, T_3):** Provided the function signature and docstring: *"You are an expert Python QA engineer. Write a suite of comprehensive unit tests to thoroughly verify the behavior of the following function based ONLY on its specification... Write 3 to 6 distinct unit test assertions or test functions... Do NOT define, stub, or re-implement the function under test... Prefer simple assert statements or standalone def test_...(): assert ... functions. Do NOT use unittest.TestCase or unittest.main()."*
- **Code-First Prompt (T_1, T_2):** Provided the signature, docstring, and faulty implementation code under the header *"Implementation under Test: ..."*, followed by identical instructions requesting 3 to 6 assertions.

3.3 Execution Environment & Evaluator Implementation

All test executions were performed in an isolated ephemeral Docker container (`python:3.11-slim`) with a default per-run timeout of **6.0 seconds**. The evaluator implements a multi-stage slicing pipeline:

1. **AST Sanitization:** The raw test output is parsed into an abstract syntax tree. Colliding imports, re-definitions of the target function, and invocations of `unittest.main()` are stripped to prevent early script termination.
2. **Test-Case Slicing:** The suite is decomposed into individual standalone test cases $\{t_1, \ldots, t_k\}$ (standalone test functions, test class methods, or top-level assertion blocks).
3. **Reference Execution (C_{ref}):** Each test case t_j is executed individually against the reference implementation C_{ref}. Test cases passing 100% on C_{ref} are retained in the valid subset T_{valid}.
4. **Oracle Validity Definition (V):** In accordance with the evaluator implementation, a test suite is classified as a *usable valid oracle* ($V = 1$) if and only if it produces **at least one valid test case** ($|T_{\text{valid}}| \geq 1$). A suite is considered *strictly clean* on reference if all generated test cases pass on C_{ref} ($|T_{\text{valid}}| = k$).

Table 2: Overall 2×2 Factorial Summary Across All 900 Evaluations ($N = 225$ Matched Trials).

Condition	Code Exposure	Model Identity	Evaluations	Usable Suites ($\lvert T_{\text{valid}} \rvert \geq 1$)	Usable Rate	Detections	Detection Rate
T_0	Specification Only	Same Model (Coder)	225	194	86.2%	76	**33.78%**
T_1	Code-First Exposure	Same Model (Coder)	225	146	64.9%	55	**24.44%**
T_2	Code-First Exposure	Cross-Family Model	225	192	85.3%	41	**18.22%**
T_3	Specification Only	Cross-Family Model	225	218	96.9%	90	**40.00%**

5. **Defect Detection (D):** If $V = 0$ (no valid test cases), defect detection is marked as $D = 0$. If $V = 1$, the valid test cases T_{valid} are executed individually against the faulty implementation C_{faulty}. A defect is marked as detected ($D = 1$) if at least one valid test case triggers an `AssertionError`, an unhandled runtime exception, or exceeds the 6.0-second timeout.

Across the 225 repeated trial quadruplets, 900 test suites were evaluated. Because each test case was executed individually on both reference and faulty implementations (alongside compilation checks), the experiment comprised several thousand isolated sandbox executions.

3.4 Statistical Analysis Framework

Because repeated trials are nested within 15 problem clusters, observations violate independent-and-identically-distributed assumptions. Our primary analytical model is a hierarchical **Generalized Linear Mixed Model (GLMM)** with a binomial logit link, fitted via maximum likelihood with adaptive Gauss-Hermite quadrature ($nAGQ = 9$) using `lme4` [1]:

$$\begin{aligned} \text{logit}\left(P(D_{ij} = 1)\right) = \beta_0 &+ \beta_1 \text{Exposure}_{ij} + \beta_2 \text{CrossModel}_{ij} \\ &+ \beta_3 (\text{Exposure}_{ij} \times \text{CrossModel}_{ij}) \\ &+ \sum_k \gamma_k \text{Coder}_{k,i} + u_i \end{aligned} \tag{1}$$

where D_{ij} is the defect detection indicator for trial j on problem i; Exposure $\in \{0, 1\}$ encodes code exposure; CrossModel $\in \{0, 1\}$ encodes architectural diversity; Coder captures coder fixed effects; and $u_i \sim \mathcal{N}(0, \sigma^2_{\text{problem}})$ represents the problem-specific random intercept.

As supporting non-parametric checks, we evaluate paired contingency tables using exact two-sided McNemar tests [9] and assess statistical equivalence using Two One-Sided Tests (TOST) [5] with an equivalence bound $\theta = \pm 0.08$.

4 Empirical Results

A total of 900 condition evaluations across 225 matched trial quadruplets were recorded. Table 2 summarizes the overall performance.

4.1 RQ1: Code-First Exposure Effect

In the matched trial-level paired comparison (T_0 vs. T_1), defect detection dropped from 33.78% (76/225) in the specification-only baseline down to 24.44% (55/225) under code exposure—an absolute difference of 9.33 percentage points.

Table 3 details the 2×2 paired contingency matrix. The comparison yielded 51 discordant pairs: in 36 trials, T_0 detected the defect while T_1 missed it; in 15 trials, T_1 detected while T_0 missed. The two-sided exact McNemar test confirms this paired difference:

$$\text{Exact } p = 0.0046, \quad \text{Matched OR} = 2.400\ [1.314, 4.383]$$

(The asymptotic continuity-corrected statistic is $\chi^2 = 7.843, p = 0.0051$).

Table 3: Paired Contingency Matrix for RQ1 (T_0 vs. T_1, $N = 225$).

	T_1 **Detected**	T_1 **Missed**	**Row Total**
T_0 **Detected**	40	36	76
T_0 **Missed**	15	134	149
Column Total	55	170	225

4.2 RQ2: Cross-Family Diversity & Equivalence under Code

Holding code exposure constant (T_1 vs. T_2), the same-family model detected defects in 24.44% (55/225) of trials, whereas the cross-family model detected defects in 18.22% (41/225) of trials (a difference of 6.22 percentage points).

Table 4 shows the paired contingency matrix (32 vs. 18 discordant pairs). The exact McNemar test indicates that the difference is not statistically significant at the 5% level ($p = 0.0649$). Applying TOST equivalence testing with margin $\theta = \pm 0.08$, the 90% confidence interval for the difference in paired proportions is $[0.0110, 0.1135]$. Because the upper bound exceeds the +8% margin, the TOST null hypothesis cannot be rejected ($p_{\text{TOST}} = 0.2841$). **Conclusion:** Performance between same-family and cross-family generators under code exposure cannot be demonstrated as statistically equivalent within an 8% margin.

Table 4: Paired Contingency Matrix for RQ2 (T_1 vs. T_2, $N = 225$).

	T_2 **Detected**	T_2 **Missed**	**Row Total**
T_1 **Detected**	23	32	55
T_1 **Missed**	18	152	170
Column Total	41	184	225

4.3 RQ3: Factorial Interaction Analysis (GLMM)

Table 5 presents the parameter estimates of the protocol GLMM. The model fit exhibits no convergence warnings (optimizer code 0, problem random intercept standard deviation $\sigma = 1.9223$).

Table 5: Hierarchical Mixed Logistic Regression Parameter Estimates ($N = 900$, 15 Problems, $nAGQ = 9$).

Fixed Effect Term	**Estimate**	**SE**	z**-value**	p**-value**	**OR [95% CI]**
(Intercept)	-0.5051	1.3844	-0.365	0.7152	0.604 [0.040, 9.100]
`exposure`	-0.7402	0.2690	-2.752	**0.0059**	0.477 [0.282, 0.808]
`cross_model`	0.4506	0.2550	1.767	0.0772	1.569 [0.952, 2.587]
`coder: Llama-3.3-70B`	-0.9878	1.5168	-0.651	0.5149	0.372 [0.019, 7.281]
`coder: Qwen-2.5-7B`	0.8309	2.3802	0.349	0.7270	2.295 [0.022, 243.7]
`coder: Qwen-Coder-32B`	0.1484	1.9505	0.076	0.9394	1.160 [0.025, 53.05]
`exposure:cross_model`	**-1.0358**	**0.3887**	**-2.665**	**0.0077**	**0.355 [0.166, 0.760]**

The interaction term β_3(Exposure × CrossModel) is negative and statistically significant ($\beta_3 = -1.0358, p = 0.00770, \text{OR} = 0.3549$). Furthermore, a Likelihood-Ratio Test comparing

the full model against an additive model confirms that the interaction improves model fit ($\chi^2 = 7.213, \text{df} = 1, p = 0.00724$).

Substantive Finding: The observed benefit of changing the model family differs depending on prompt exposure. Under specification-only conditions, switching to an independent model family produces an increase in detection ($T_3 = 40.00\%$ vs. $T_0 = 33.78\%$). However, under code exposure, cross-model diversity does not provide a compensatory benefit ($T_2 = 18.22\%$).

5 Descriptive Examination: Test Usability vs. Cognitive Framing

A prominent question in prior literature is whether detection reductions under code exposure represent cognitive "confirmation bias" or operational degradation in test suite validity.

5.1 Usable Suite Conditioning

As detailed in Table 2, code exposure is accompanied by a marked decline in the proportion of test suites that execute cleanly against the reference implementation C_{ref}:

- T_0 (Spec-Only) achieved a usable suite rate of 86.2% (194/225).
- T_1 (Code-First) dropped to a usable suite rate of 64.9% (146/225).

When calculating defect detection rates conditionally among usable suites only (i.e., suites where $|T_{\text{valid}}| \geq 1$):

- T_0 detection among usable suites: $76/194 = \mathbf{39.18}\%$.
- T_1 detection among usable suites: $55/146 = \mathbf{37.67}\%$.
- T_2 detection among usable suites: $41/192 = \mathbf{21.35}\%$.
- T_3 detection among usable suites: $90/218 = \mathbf{41.28}\%$.

Conditioning on test usability selects different subsets in each experimental condition and does not substitute for the primary end-to-end outcome. Descriptively, however, the gap between T_0 and T_1 among usable suites (39.18% vs. 37.67%) is substantially narrower than the 9.33 percentage point end-to-end difference.

5.2 Methodological Interpretation

These descriptive proportions indicate that attributing the entire exposure penalty to a cognitive mechanism overextends the evidence. Rather, a substantial contributing factor is that **code exposure reduces the likelihood of generating a valid test suite that adheres to the reference specification**. When exposed to implementation code, models frequently generate assertions tailored to internal variable states or local syntactic structures of the faulty code, resulting in tests that fail against C_{ref}.

Thus, the overall reduction in fault detection under code exposure reflects an operational breakdown in oracle validity, rather than a purely psychological confirmation effect.

6 Exploratory Sensitivity Analysis & Task Heterogeneity

To examine whether findings are sensitive to problem clustering, we report two exploratory sensitivity analyses.

6.1 Problem-Level Aggregated Contrasts

We collapsed the 225 trials into 15 cluster means (one per problem) and applied paired t-tests across the 15 problem-level differences (df = 14). Table 6 presents the contrasts.

Table 6: Exploratory Problem-Level Aggregated Contrasts ($N = 15$ Problems, df = 14).

Contrast	**Mean Difference**	**95% Confidence Interval**	p**-value**
$T_0 - T_1$ (Exposure Effect, Same Coder)	+0.0933	$[-0.0468, 0.2334]$	0.1750
$T_1 - T_2$ (Diversity under Code Exposure)	+0.0622	$[-0.1112, 0.2356]$	0.4544
$T_3 - T_2$ (Exposure under Cross-Model)	+0.2178	$[-0.0320, 0.4676]$	0.0826
Probability Interaction Contrast	**-0.1244**	**[-0.4017, 0.1528]**	**0.3521**

While the mean difference point estimates remain identical to the trial-level analysis ($T_0 - T_1 = 9.33$ pp), the 95% confidence intervals span zero when aggregated across 15 problem clusters. Furthermore, the probability-scale interaction contrast (-12.44 pp, $p = 0.3521$) is inconclusive; this metric measures linear probability differences, differing conceptually from the conditional log-odds interaction estimated by the GLMM.

6.2 Random-Slopes GLMM

We also fitted an exploratory mixed model incorporating problem-specific random slopes for exposure, diversity, and interaction. In this random-slopes specification, the interaction coefficient shifted to $\beta_3 = -7.2365$ ($SE = 3.3689, z = -2.148, p = 0.0317$). While the interaction retained nominal statistical significance at the 5% level, the substantial shift in coefficient magnitude underscores large task-level slope variance and wide uncertainty regarding effect magnitude across diverse software problems.

7 Discussion & Practical Scope

Within the scope of the evaluated Python algorithmic tasks and model architectures, our results suggest specific considerations for automated testing workflows:

1. **Decouple Test Synthesis from Implementation Inspection:** Providing implementation code directly in the prompt of an LLM test generator does not enhance defect detection and reduces test oracle validity. Where formal specifications, docstrings, or interface signatures are available, generating tests from specifications alone provides higher net fault discovery.

2. **Position Model Diversity at the Specification Stage:** In multi-agent testing setups, cross-model ensembles provide the largest observed benefit when generating black-box tests (T_3 achieved 40.00% detection). Conversely, deploying an independent model to inspect existing code (T_2) does not compensate for code-exposure penalties.

8 Threats to Validity

8.1 Internal Validity

- **Provider Routing & Execution Continuity:** LLM invocations were routed through the OpenRouter unified API. Dynamic provider routing and fallbacks occurred during evaluation; hence, strict hardware-level provider pinning cannot be claimed. Transient credit exhaustion caused temporary HTTP 402 pauses; however, all 900 evaluations in the

final dataset were completed under identical prompts and evaluated in Docker sandboxes with clean status recording.

8.2 External Validity

- **Language and Task Generalizability:** The study is scoped to 15 standalone algorithmic Python tasks. Generalizability to object-oriented frameworks, database systems, or other programming languages (such as Elixir [7]) requires broader empirical replication.
- **Model Versioning:** Evaluated open-weight models reflect weights available in late 2024. While open weights ensure scientific reproducibility, newer model generations may exhibit different generation dynamics.

8.3 Construct Validity

- **Absence of Cognitive Measurements:** LLMs are probabilistic sequence predictors. We explicitly make no claims regarding human cognitive states or mental confirmation bias; all findings represent functional input-output oracle validity and defect detection metrics.

9 Conclusion

This paper presented a 2×2 factorial empirical investigation ($N = 225$ matched trial quadruplets, 900 condition evaluations) disentangling Code Exposure from Model Diversity in LLM test generation. Using a hierarchical GLMM on verified natural Python logic defects, we confirmed a statistically significant negative interaction ($\beta = -1.0358, p = 0.0077, \text{OR} = 0.355$). While cross-model diversity achieves superior fault detection under specification-only conditions (40.00%), code exposure suppresses this benefit (18.22%), a reduction largely driven by degraded test oracle validity (86.2% vs. 64.9%). Within the studied tasks and models, our findings demonstrate that separating test synthesis from implementation inspection is essential for maximizing test oracle quality.

CRediT Authorship Contribution Statement

Ziad Emad Allam: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data Curation, Writing – Original Draft, Writing – Review & Editing.

Declaration of Competing Interest

The author declares that he has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data Availability

The supplementary public package is openly available at urlhttps://github.com/ziademad02153/llm-test-bias-paper. It contains all 900 recorded condition outcomes across 225 matched trials on 15 tasks and the R scripts and outputs needed to reproduce the statistical analyses. Source implementations, task specifications and generated test suites are supplied for nine tasks with documented licenses. Source-bearing artifacts for six tasks (REAL_TASK_004, 006, 010, 011, 019 and 020) are withheld pending clarification of redistribution permission. Their numerical outcomes and source repository/commit pointers remain included. Consequently, the public package supports reanalysis of the full study but does not support complete re-execution of test evaluation for the six withheld tasks. No numerical outcomes were removed or recalculated for this release.

Declaration of Generative AI in Scientific Writing

During the preparation of this work, the author utilized large language models (specifically Claude 3.7 Sonnet and Antigravity AI coding assistant) for drafting portions of the text, writing automated test-execution scripts, and structuring the statistical modeling pipelines. All generated text, scripts, and statistical models were rigorously reviewed, independently audited, and verified against raw experimental logs and Docker execution traces by the author, who takes full intellectual, methodological, and empirical responsibility for the contents and conclusions of this publication.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

References

[1] Douglas Bates, Martin Mächler, Benjamin Bolker, and Steve Walker. Fitting linear mixed-effects models using `lme4`. *Journal of Statistical Software*, 67(1):1–48, 2015.

[2] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[3] René Just, Darioush Jalali, Laura Inozemtseva, Michael D. Ernst, Reid Holmes, and Gordon Fraser. Are mutants a valid substitute for real faults in software testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE 2014)*, pages 654–665, 2014.

[4] Michael Konstantinou, Florian Tambon, and Mike Papadakis. On the risk of coding before testing: An empirical study on LLM-based test generation workflow. *arXiv preprint arXiv:2607.05139*, 2026.

[5] Daniël Lakens. Equivalence testing for psychological research: A tutorial on two one-sided tests (TOST). *Social Psychological and Personality Science*, 8(4):355–362, 2017.

[6] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Koushik Sen. CODAMOSA: Escaping coverage plateaus in test generation with pre-trained large language models. In *Proceedings of the 45th International Conference on Software Engineering (ICSE 2023)*, pages 919–931, 2023.

[7] Noah Leu, Julian Oertel, and Regina Hebig. Don't bother to use a second LLM and write tests yourself! a study on Elixir. In *Proceedings of the 20th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM 2026)*, volume 394 of *LIPIcs*, pages 11:1–11:20, 2026.

[8] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems (NeurIPS 2023)*, volume 36, pages 21558–21572, 2023.

[9] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

[10] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering*, 50(1):85–105, 2024.